- Newsletter

Search Site

Go

developer.com

Login | Register

webreference™
dev the web

Subscribe

Enter your Email Addres

Close

Basic SOA Using REST: A Hands-On Guide to Implementing Web services and Service Oriented Architecture

Accusoft's OCR Xpress offers a value-based OCR solution, providing a quick and easy way to convert document images to searchable PDFs or text for document indexing. Try it for free today.

Sponsored by Accusoft

[next]

Basic SOA Using REST

By Mark Hansen

Digg This Add to del.icio.us

In this chapter, I describe the basic tools and techniques for implementing SOA components using the REST paradigm. REST stands for Representational State Transfer. It was first introduced by Roy Fielding in his 2000 doctoral dissertation (Fielding is one of the principal authors of the HTTP specification and a co-founder of the Apache HTTP Server project.). For the past several years, a great debate has been going on about the merits of the REST versus SOAP architectural styles for Web Services. It is not my intention, in this book, to weigh in on either side of that debate. My feeling is that both approaches are useful for implementing SOA components. For simple applications, REST is an easy way to get started.

If you are an advanced Java programmer, you might find the first half of this chapter to be very basic. I have intentionally started out with simplistic examples, using only HTTP and servlets, so that readers who are not advanced in Java can come up the learning curve and get a sense of the basics before introducing the Java Web Services (JWS) APIs. If you have a good grounding in HTTP and Java servlets, please feel free to skip the introductory material and focus on the sections dealing with JAX-WS.

3.1 Why REST?

Some readers may wonder why this book starts with REST before discussing SOAP and WSDL-based Web Services. The reason is that REST is easy to understand. By starting with REST, I can describe some of the basic SOA Web Services concepts without getting into the complexities of SOAP and WSDL. Also, the limitations of REST provide the motivation for introducing SOAP and WSDL in Chapter 4. If you are not interested in REST, feel free to skip ahead to Chapter 4.

3.1.1 What Is REST?

REST-style services (i.e., RESTful services) adhere to a set of constraints and architectural principles that include the following:

- RESTful services are *stateless*. As Fielding writes in Section 5.1.3 of his thesis, "each request from client to server must contain all the information necessary to understand the request, and cannot take advantage of any stored context on the server."
- RESTful services have a uniform interface. This constraint is usually taken to mean that the only allowed operations are the HTTP operations: GET, POST, PUT, and DELETE.
- REST-based architectures are built from resources (pieces of information) that are uniquely identified by URIs. For example, in a RESTful purchasing system, each purchase order has a unique URI.

- REST components manipulate resources by exchanging representations of the resources. For example, a purchase order resource can be represented by an XML document. Within a RESTful purchasing system, a purchase order might be updated by posting an XML document containing the changed purchase order to its URI.

Fielding writes that "REST-based architectures communicate primarily through the transfer of representations of resources" (Section 5.3.3). This is fundamentally different from the Remote Procedure Call (RPC) approach that encapsulates the notion of invoking a procedure on the remote server. Hence, RPC messages typically contain information about the procedure to be invoked or action to be taken. This information is referred to as a verb in a Web service request. In the REST model, the only verbs allowed are GET, POST, PUT, and DELETE. In the RPC approach, typically many operations are invoked at the same URI. This is to be contrasted with the REST approach of having a unique URI for each resource.

These are the basic principles behind REST. However, when people talk about the benefits of RESTful systems today, they usually are not strictly applying these principles. For example, among REST advocates, keeping shopping cart data on the server and maintaining a session related to the shopping process that is using the cart is acceptable. (Storing session information or shopping cart data on the server is a clear violation of Fielding's original REST concept since it violates the requirement that a service be stateless.) In fact, the XML/HTTP Binding provided by JAX-WS for implementing RESTful services provides for session management capabilities using cookies, URL rewriting, and SSL session IDs.

More significant deviations from Fielding's definition of REST involve getting around the "uniform interface" constraint by embedding verbs and parameters inside URLs. The Amazom.com REST interface, for example, includes verbs in query strings and doesn't have unique URIs for each resource. Systems like this, although labeled as RESTful, are really starting to look very much like RPC using XML over HTTP without SOAP.

For the purposes of this book, I am not going to wade into a debate on what is or isn't RESTful. I simply define RESTful Web Services in contrast to SOAP Web Services. Table 3Â1 illustrates the principal differences.

Table 3–1 RESTful Web Services versus SOAP Web Services

	REST	SOAP
Message Format	XML	XML inside a SOAP Envelope
Interface Definition	none[a]	WSDL
Transport	HTTP	HTTP, FTP, MIME, JMS, SMTP, etc.

a. Some would argue that XML Schema could be used as an interface definition for RESTful services. Not only is that approach possible, but it is used in many practical cases. However, it is not a complete interface solution because many, if not most, RESTful services incorporate HTTP parameters (e.g., URL query strings) in addition to XML as part of their invocation interface. Chapter 9 looks at the Yahoo! Shopping RESTful interface, which uses HTTP parameters in this manner.

This is consistent with common usage in the REST versus SOAP debates. REST uses simple XML over HTTP without a WSDL interface definition.

3.1.2 Topics Covered in This Chapter

In addition to introducing RESTful Web Services, this chapter introduces and reviews some basic techniques for integrating Enterprise Information Systems (EISs) using XML, XSLT, HTTP, and Java. For each example, I demonstrate how to implement it with and without JWS. The versions of the examples without JWS use basic Java HTTP and XML techniques. Both approaches are provided to give you a sense of what is really happening, under the covers, when a Web service is consumed or deployed using JWS. This should give you a better understanding of the mechanisms underlying JWS and when to use them. For simple Web services, often it is easier to work with the basic Java tools than to pull out all the power of JWS. On the other hand, you will see from these examples how things can quickly get complicated and require the power of the JWS technologies.

Since one focus of this book is on SOA-style development for the enterprise, many of the examples deal with EISÂthe basic infrastructure of most corporate computing environments. This chapter describes

- Structuring EIS Records as XML documents
- Getting EIS records from a REST service (with and without JWS)
- Posting EIS records to a REST service (with and without JWS)
- Basic SOA-style integration of REST services using XSLT for data transformation
- Deploying a REST service to be used for getting EIS recordsÂin other words, an HTTP GET service (with and without JWS)
- Deploying a REST service to be used for posting EIS recordsÂin other words, an HTTP POST service (with and without JWS)

[next]

URL: http://www.webreference.com/programming/basic_soa/index.html